

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 1 – Test Case Design Assignment

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 1 – Test Case Design Assignment
Weightage:	20%	Total Marks:	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	VIVETHA B	Reg. No.:	192424170
Date:		Duration:	60 minutes

Code of Conduct

I VIVETHA B 192424170 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: VIVETHA B

Criteria	Mark
Requirement & Test Scenario Identification (15%)	/5
Test Case Design & Completeness (25%)	/4
Test Data & Expected Results (20%)	/4
Application of Test Design Techniques (15%)	/4
Traceability & Test Coverage (15%)	/4
Documentation & Presentation (10%)	/4
TOTAL	/25

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Test Scenario Identification(15%)	Clearly identifies all relevant functional/non-functional requirements and comprehensive test scenarios	Identifies most requirements and scenarios	Identifies basic requirements and scenarios	Requirements/scenarios are incomplete or unclear
Test Case Design & Completeness(25%)	Comprehensive test cases covering normal, boundary, invalid, and exceptional conditions	Covers most important conditions with minor gaps	Covers basic conditions but misses several important cases	Test cases are very limited or incomplete
Test Data & Expected Results(20%)	Appropriate test data with precise, measurable expected results for every test case	Mostly appropriate test data and expected results	Some test data/results are unclear or incomplete	Test data and expected results are largely missing/incorrect
Application of Test Design Techniques(15%)	Correctly applies multiple techniques such as Equivalence Partitioning, Boundary Value Analysis, Decision Table, and State Transition	Correctly applies at least two relevant techniques	Applies one technique with limited justification	Techniques are not applied or incorrectly applied
Traceability & Test Coverage(15%)	Excellent mapping between requirements, scenarios, and test cases with strong coverage	Good traceability and coverage with minor gaps	Partial traceability and moderate coverage	Poor/no traceability and inadequate coverage
Documentation & Presentation(10%)	Well-structured, clear, professional, consistent, and easy to understand	Well-organized with minor presentation issues	Adequately organized but has some clarity/formatting issues	Poorly organized, unclear, or incomplete documentation

PHISHING EMAIL DETECTION AND CYBER THREAT INTELLIGENCE SYSTEM

1. Identify the **functional and non-functional requirements** to be tested.

Functional Requirements

These describe **what the system should do**.

- **User Login and Authentication** – Verify that authorized users can securely log in and unauthorized users are denied access.
- **Email Input/Upload** – Test whether the system accepts email text, email files, or uploaded email datasets correctly.
- **Email Preprocessing** – Verify that the system extracts and cleans important information such as sender, subject, body, links, and attachments.
- **Phishing Email Detection** – Test whether the system correctly identifies emails as **Phishing** or **Legitimate**.
- **Threat Classification** – Verify whether detected threats are classified into categories such as phishing, malware, spam, credential theft, or suspicious links.
- **URL Analysis** – Test whether URLs contained in emails are analyzed for suspicious characteristics.
- **Sender Analysis** – Verify whether the system checks sender information, domain reputation, and suspicious sender patterns.
- **Threat Intelligence Integration** – Test whether the system retrieves relevant threat information from threat-intelligence sources.
- **Risk Score Generation** – Verify that the system generates an appropriate risk or threat score for each analyzed email.
- **Alert Generation** – Test whether users receive alerts when a high-risk or malicious email is detected.
- **Result Display** – Verify that detection results, threat category, risk score, and reasons for detection are displayed correctly.
- **Explainable Detection** – Test whether the system provides understandable reasons for classifying an email as phishing.

- **Threat History** – Verify that previous email-analysis results are stored and can be viewed by authorized users.
- **Search and Filtering** – Test whether users can search and filter detected threats based on date, threat type, risk level, or sender.
- **Report Generation** – Verify that the system can generate reports containing detection results and threat intelligence information.

Non-Functional Requirements

These describe **how well the system should perform**.

- **Performance** – The system should analyze an email and provide results within an acceptable response time.
- **Accuracy** – The phishing detection model should correctly identify phishing and legitimate emails while minimizing false positives and false negatives.
- **Security** – User credentials, email content, threat data, and system information should be protected from unauthorized access.
- **Reliability** – The system should provide consistent results and continue functioning correctly without frequent failures.
- **Availability** – The detection system should be available whenever authorized users need to analyze emails.
- **Scalability** – The system should be capable of handling an increasing number of emails and users without major performance degradation.
- **Usability** – The interface should be simple, clear, and easy for users to understand.
- **Maintainability** – The system should allow developers to update detection models, threat-intelligence sources, and system components easily.
- **Compatibility** – The application should work correctly across supported browsers, operating systems, and common email formats.
- **Privacy** – Sensitive email content and user information should be handled securely and should not be unnecessarily exposed.
- **Response Time** – Detection, URL analysis, and threat-intelligence queries should return results within the defined time limit.
- **Fault Tolerance** – The system should handle invalid emails, unavailable threat-intelligence services, network failures, and corrupted files without crashing.

- **Auditability** – Important user actions, detection events, and security-related activities should be recorded for auditing.
- **Data Integrity** – Stored detection results and threat-intelligence information should remain accurate and protected from unauthorized modification.
- **Extensibility** – The system should support adding new phishing patterns, machine-learning models, and threat-intelligence sources in the future.

2. Identify the **test scenarios** for the assigned problem statement.

- Test whether a valid user can log in successfully.
- Test whether an invalid username or password is rejected.
- Test whether the system accepts a valid email for analysis.
- Test whether the system handles empty email input correctly.
- Test whether the system accepts supported email file formats.
- Test whether the system rejects unsupported or corrupted email files.
- Test whether email preprocessing correctly extracts the sender, subject, body, URLs, and attachments.
- Test whether a legitimate email is correctly classified as **Legitimate**.
- Test whether a known phishing email is correctly classified as **Phishing**.
- Test whether suspicious URLs inside an email are detected correctly.
- Test whether suspicious sender addresses or domains are identified.
- Test whether malicious attachments are detected or flagged.
- Test whether the system generates an appropriate **risk score** for analyzed emails.
- Test whether the system correctly categorizes different types of cyber threats.
- Test whether threat-intelligence information is retrieved correctly.
- Test whether the system displays meaningful reasons for identifying an email as phishing.
- Test whether high-risk emails generate appropriate security alerts.
- Test whether analyzed email results are stored correctly in the database.
- Test whether users can search and filter previous threat-analysis results.
- Test whether authorized users can generate and download threat reports.
- Test whether unauthorized users are prevented from accessing protected threat information.
- Test whether the system handles network or threat-intelligence API failures without crashing.
- Test whether the system provides results within the expected response time.

- Test whether sensitive email and user information is protected from unauthorized access.
 - Test whether the system maintains accurate logs of detection and security-related activities.
 - Test whether the system handles large email files and large datasets correctly.
 - Test whether the system works correctly across supported browsers and devices.
 - Test whether the system continues functioning correctly after errors, invalid inputs, or temporary service failures.
3. Prepare detailed **test cases** covering normal, boundary, invalid, and exceptional conditions.

Normal Conditions

TC01 – Valid Login:

Verify that a registered user can log in successfully using valid username and password.

TC02 – Valid Email Upload:

Verify that the system accepts and uploads a valid email file in the supported format.

TC03 – Valid Email Input:

Verify that the system accepts properly formatted email content and processes it successfully.

TC04 – Legitimate Email Detection:

Verify that a genuine and safe email is correctly classified as legitimate.

TC05 – Phishing Email Detection:

Verify that an email containing phishing characteristics is correctly identified as phishing.

TC06 – Suspicious URL Detection:

Verify that suspicious or malicious URLs present in an email are detected and flagged.

TC07 – Sender Analysis:

Verify that suspicious sender addresses or domains are identified by the system.

TC08 – Risk Score Generation:

Verify that the system calculates and displays an appropriate risk score for the analyzed email.

TC09 – Detection Explanation:

Verify that the system provides understandable reasons for classifying an email as phishing.

TC10 – Threat Report Generation:

Verify that the system generates an accurate report containing the email analysis and threat information.

Boundary Conditions

TC11 – Empty Email Body:

Verify that the system properly handles an email with an empty body and displays a suitable validation message.

TC12 – Minimum Email Content:

Verify that the system can process an email containing only the minimum required information.

TC13 – Maximum Email Size:

Verify that an email at the maximum allowed size is accepted and processed correctly.

TC14 – Email Size Exceeded:

Verify that the system rejects an email that exceeds the permitted file or message size.

TC15 – Maximum Subject Length:

Verify that the system correctly accepts a subject containing the maximum permitted number of characters.

TC16 – Subject Length Exceeded:

Verify that the system handles a subject that exceeds the maximum character limit.

TC17 – Maximum URLs:

Verify that the system correctly analyzes an email containing the maximum supported number of URLs.

TC18 – Large Dataset:

Verify that the system can process a dataset containing the maximum supported number of email records.

TC19 – Risk Score Boundary:

Verify that minimum and maximum risk scores are correctly calculated and assigned to the appropriate risk levels.

TC20 – Maximum Attachments:

Verify that the system correctly processes an email containing the maximum allowed number of attachments.

Invalid Conditions**TC21 – Invalid Login:**

Verify that the system rejects login attempts made with incorrect username or password.

TC22 – Empty Username:

Verify that the system displays a validation message when the username field is left empty.

TC23 – Empty Password:

Verify that the system displays a validation message when the password field is left empty.

TC24 – Invalid Email Format:

Verify that incorrectly formatted email addresses are rejected by the system.

TC25 – Unsupported File:

Verify that unsupported or potentially dangerous file formats are rejected during upload.

TC26 – Corrupted Email File:

Verify that the system detects a corrupted email file and displays an appropriate error message.

TC27 – Empty File:

Verify that the system rejects an uploaded file containing no usable email information.

TC28 – Invalid URL:

Verify that incorrectly formatted URLs are identified and handled safely.

TC29 – Special Characters:

Verify that unexpected or potentially harmful special characters are safely handled without affecting the system.

TC30 – Invalid Threat Data:

Verify that the system safely handles incorrect or incomplete data received from a threat-intelligence source.

Exceptional Conditions**TC31 – API Unavailable:**

Verify that the system handles the unavailability of the external threat-intelligence API without crashing.

TC32 – Network Failure:

Verify that the system displays an appropriate error when the network connection is interrupted during analysis.

TC33 – Database Failure:

Verify that the system handles database connection failure without losing or corrupting existing information.

TC34 – ML Model Failure:

Verify that the system handles machine-learning model failure and provides an appropriate error message.

TC35 – Multiple Requests:

Verify that the system remains stable when many users submit email-analysis requests simultaneously.

TC36 – Server Failure:

Verify that the system handles server failure and recovers properly when the service becomes available again.

TC37 – Session Timeout:

Verify that inactive user sessions expire automatically after the configured time for security.

TC38 – Unauthorized Access:

Verify that unauthorized users cannot access restricted threat information or reports.

TC39 – Malicious Input:

Verify that malicious scripts or injection inputs are detected, sanitized, and prevented from executing.

TC40 – API Timeout:

Verify that the system stops waiting after the defined API timeout and displays an appropriate message.

TC41 – Duplicate Email:

Verify that the system correctly handles the same email being submitted multiple times.

TC42 – Invalid Model Output:

Verify that the system safely handles unexpected or invalid results returned by the detection model.

TC43 – Storage Failure:

Verify that the system properly handles failures while saving analysis results.

TC44 – Sudden Shutdown:

Verify that unexpected application shutdown does not corrupt stored email or threat-analysis data.

TC45 – Invalid Risk Score:

Verify that the system detects and safely handles a risk score outside the permitted range.

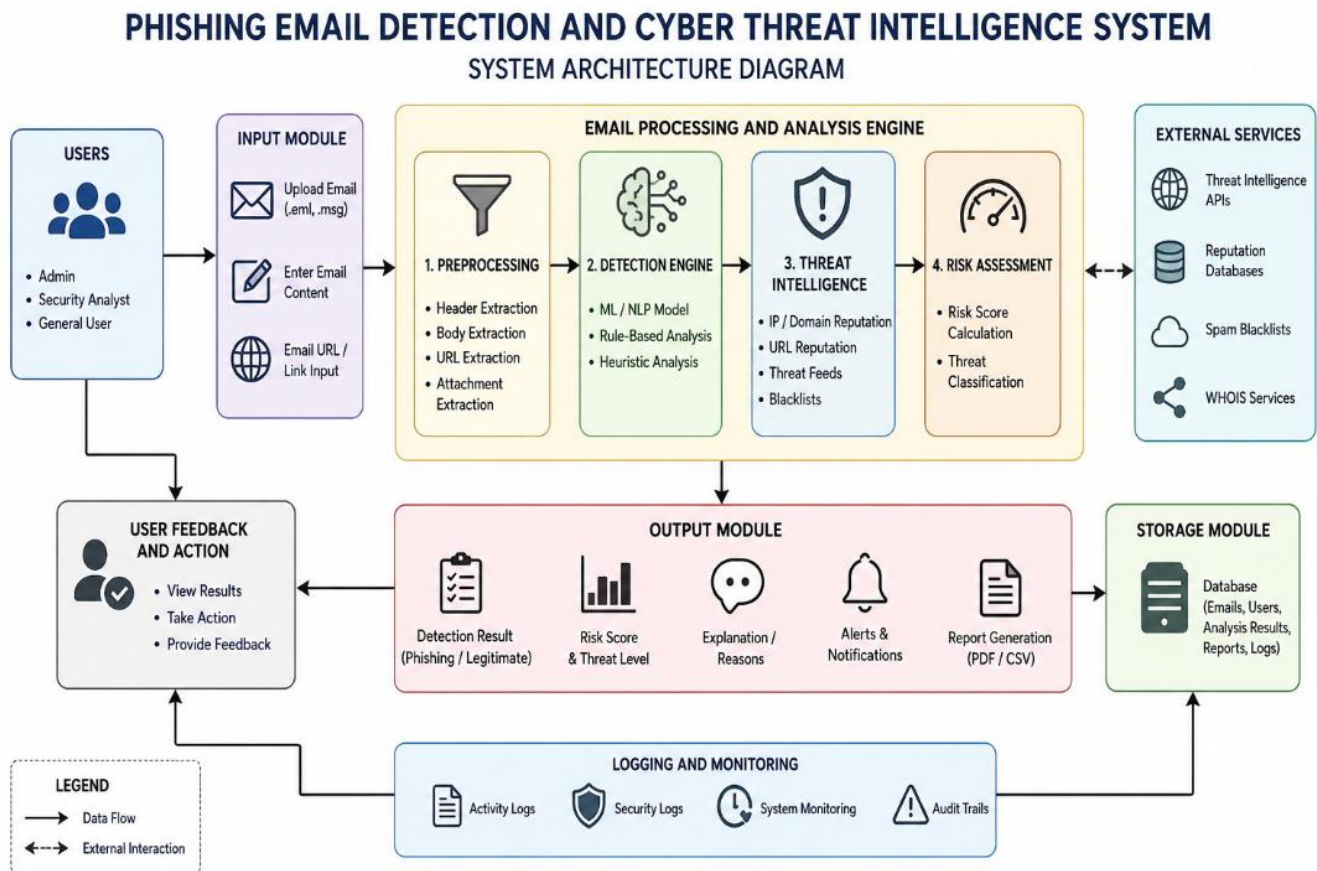
4. Include **Test Case ID, Test Scenario, Test Steps, Test Data, Expected Result, Actual Result, and Pass/Fail Status.**

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Pass/Fail Status
TC01	Valid Login	Enter username and password, then click Login.	Valid credentials	User should log in successfully.	To be filled during testing.	Not Executed
TC02	Invalid Login	Enter incorrect credentials and click Login.	Wrong username/password	Login should be rejected with an error message.	To be filled during testing.	Not Executed
TC03	Upload Valid Email	Select and upload a valid email file.	Valid .eml file	Email should be uploaded successfully.	To be filled during testing.	Not Executed
TC04	Phishing Detection	Upload a known phishing email and start analysis.	Phishing email	System should identify it as phishing.	To be filled during testing.	Not Executed
TC05	Legitimate Email Detection	Upload a genuine email and analyze it.	Legitimate email	System should classify it as legitimate.	To be filled during testing.	Not Executed

TC06	Suspicious URL Detection	Submit an email containing a suspicious URL.	Malicious URL	URL should be detected and flagged.	To be filled during testing.	Not Executed
TC07	Sender Analysis	Analyze an email from a suspicious domain.	Suspicious sender	Sender/domain should be flagged.	To be filled during testing.	Not Executed
TC08	Risk Score	Analyze an email containing phishing indicators.	Suspicious email	Appropriate risk score should be generated.	To be filled during testing.	Not Executed
TC09	Threat Classification	Submit an email containing malicious content.	Malware/phishing email	Correct threat category should be displayed.	To be filled during testing.	Not Executed
TC10	Detection Explanation	Analyze a phishing email and view the result.	Phishing email	System should display reasons for detection.	To be filled during testing.	Not Executed
TC11	Empty Email	Submit an empty email input.	Blank input	System should display a validation message.	To be filled during testing.	Not Executed
TC12	Maximum Email Size	Upload an email at the maximum allowed size.	Maximum-size email	System should accept and process it.	To be filled during testing.	Not Executed
TC13	Oversized Email	Upload an email exceeding the size limit.	Oversized file	System should reject the file.	To be filled during testing.	Not Executed
TC14	Invalid File	Upload an unsupported file format.	.exe file	System should reject the file safely.	To be filled during testing.	Not Executed
TC15	Corrupted File	Upload a damaged email file.	Corrupted .eml	System should show an appropriate error.	To be filled during testing.	Not Executed
TC16	Invalid Email Format	Enter an incorrectly formatted	abc@	System should reject the input.	To be filled	Not Executed

		email address.			during testing.	
TC17	Threat Intelligence API Failure	Analyze an email when the API is unavailable.	API unavailable	System should handle the error without crashing.	To be filled during testing.	Not Executed
TC18	Network Failure	Disconnect the network during analysis.	No internet connection	System should display a suitable error message.	To be filled during testing.	Not Executed
TC19	Unauthorized Access	Try to access restricted threat reports without permission.	Unauthorized user	Access should be denied.	To be filled during testing.	Not Executed
TC20	Malicious Input	Enter script or injection content into email fields.	<script> input	Input should be sanitized and not executed.	To be filled during testing.	Not Executed
TC21	API Timeout	Submit analysis when the threat API does not respond.	Delayed API response	System should stop waiting after the timeout period.	To be filled during testing.	Not Executed
TC22	Duplicate Email	Submit the same email multiple times.	Duplicate email	System should handle duplicates according to system rules.	To be filled during testing.	Not Executed
TC23	Database Failure	Analyze an email when the database is unavailable.	Database offline	System should show an error without crashing.	To be filled during testing.	Not Executed
TC24	Session Timeout	Remain inactive until the session expires.	Inactive session	User should be logged out securely.	To be filled during testing.	Not Executed
TC25	Report Generation	Analyze an email and generate a report.	Completed analysis	Accurate threat report should be generated.	To be filled during testing.	Not Executed

5. Apply appropriate **test design techniques** such as Equivalence Partitioning, Boundary Value Analysis, Decision Table, or State Transition Testing wherever applicable.



Test Design Techniques

For the **Phishing Email Detection and Cyber Threat Intelligence System**, the following test design techniques can be applied.

Equivalence Partitioning

Equivalence Partitioning divides input data into valid and invalid groups so that one value from each group can be tested.

- **Email input:** Valid email, invalid email, and empty email.
- **File format:** Supported .eml file, unsupported file, and corrupted file.
- **Risk score:** Low-risk, medium-risk, and high-risk email.
- **Login:** Valid credentials and invalid credentials.
- **URL:** Valid URL, suspicious URL, and invalid URL.

Example: Test one valid .eml file and one unsupported .exe file instead of testing every possible file format.

Boundary Value Analysis

Boundary Value Analysis tests values at and around the minimum and maximum limits.

- Test email size at **minimum allowed size**.
- Test email size at **maximum allowed size**.
- Test one value **below the minimum**.
- Test one value **above the maximum**.
- Test maximum subject length and one character beyond the limit.
- Test minimum and maximum number of URLs or attachments.

Example: If the maximum email size is 10 MB, test **9.99 MB, 10 MB, and 10.01 MB**.

Decision Table Testing

Decision Table Testing is useful when the system's output depends on multiple conditions.

For phishing detection, conditions can include:

- Suspicious sender
- Malicious URL
- Suspicious keywords
- Unknown domain
- Malicious attachment

The system can combine these conditions to determine the final result.

Example: If the sender is suspicious and the URL is malicious, the system should classify the email as **High Risk / Phishing**. If none of the indicators are present, it should classify the email as **Legitimate**.

State Transition Testing

State Transition Testing verifies how the system behaves when it moves from one state to another.

The email analysis process can have these states:

Email Uploaded → Processing → Threat Analysis → Result Generated → Report Saved

Possible transitions include:

- Valid email → Processing
- Invalid email → Error
- Processing completed → Result Generated
- High-risk email → Alert Generated
- API failure → Error State
- Retry → Processing Again

Example: When a user uploads a valid email, the system should move from **Uploaded** to **Processing** and then to **Analysis Completed**.

Error Guessing

Error Guessing can be used based on common user and system mistakes.

- Empty input
- Incorrect login credentials
- Corrupted email files
- Invalid URLs
- Duplicate emails
- Network failure
- API failure
- Database failure
- Malicious script input

These techniques help ensure that the system works correctly under **normal, boundary, invalid, and exceptional conditions**.

6. Ensure that the test cases provide adequate **requirement and functional coverage** for the assigned system.

Requirement and Functional Coverage

For the **Phishing Email Detection and Cyber Threat Intelligence System**, the test cases should cover all major functional and non-functional requirements. Each requirement must have at least one corresponding test case to ensure that no important functionality is missed.

- **User Authentication:** TC01 and TC02 cover valid and invalid login conditions.
- **Email Upload and Input:** TC03, TC11, TC14, and TC15 cover valid, empty, unsupported, and corrupted email inputs.
- **Phishing Detection:** TC04 and TC05 verify phishing and legitimate email classification.
- **URL Analysis:** TC06 verifies detection of suspicious URLs.
- **Sender Analysis:** TC07 verifies suspicious sender and domain detection.
- **Risk Assessment:** TC08 verifies risk-score generation for analyzed emails.
- **Threat Classification:** TC09 verifies correct identification of different cyber threats.
- **Explainable Detection:** TC10 verifies that reasons for phishing classification are provided.
- **Boundary Handling:** TC12 and TC13 verify maximum and exceeded email-size conditions.
- **Security:** TC19 and TC20 verify unauthorized access and malicious input handling.

- **Threat Intelligence:** TC17 and TC21 verify API failure and timeout conditions.
- **Network Reliability:** TC18 verifies system behavior during network failure.
- **Data Management:** TC22 and TC23 verify duplicate data and database failure handling.
- **Session Management:** TC24 verifies secure session timeout.
- **Report Generation:** TC25 verifies accurate generation of threat-analysis reports.

Therefore, the test cases provide **requirement coverage** by mapping each major system requirement to one or more test cases and provide **functional coverage** by testing the complete workflow from **user login → email submission → preprocessing → phishing detection → threat analysis → risk scoring → alert/result generation → report generation**. This ensures that the important features of the system are tested under normal, boundary, invalid, and exceptional conditions.

Deliverable: Submit a Test Case Design document containing the identified scenarios, test cases, test data, expected results, and test coverage based on the individual problem statement.